TP Le Fournil : mes réponses

Dorian Pachot · BTS SIO SLAM, 2e année · Bases de données avancées · parties 1 à 3 (octobre 2026)

Partie 1 : du MCD aux tables

```
-- Q1.1
-- Côté COMMANDE (1,n) : une commande contient au moins un produit et peut en contenir
plusieurs.
-- Côté PRODUIT (0,n) : un produit peut n'être dans aucune commande, ou dans plusieurs.
-- prix_unitaire est dans CONTENIR parce que c'est le prix payé le jour de la commande.
-- Le prix de PRODUIT peut changer ensuite : si on ne gardait que lui, le montant des anciennes
commandes changerait aussi.

-- Q1.2 MLD
-- categorie (_id_categorie_, libelle)
-- produit (_id_produit_, nom, prix, stock, #id_categorie)
-- client (_id_client_, nom, prenom, email, telephone, ville, date_inscription)
-- commande (_id_commande_, date_commande, date_retrait, statut, montant_total, #id_client)
-- ligne_commande (_#id_commande_, _#id_produit_, quantite, prix_unitaire)
-- (souligné = entre _ _)

-- Q1.2.1
-- id_categorie va dans produit, id_client va dans commande.
-- pour une association (1,1) - (0,n), la clé de l'entité côté 0,n devient une clé étrangère
-- dans la table de l'entité côté 1,1.

-- Q1.2.2
--CONTENIR est une association n-n (1,n / 0,n) et elle porte des attributs (quantite,
prix_unitaire),
-- donc elle devient une table. APPARTENIR et PASSER sont des 1-n : elles disparaissent, on met
juste une clé étrangère.
-- Clé primaire de ligne_commande : le couple (id_commande, id_produit).

-- Q1.4.1
CREATE DATABASE fournil CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE fournil;

-- Q1.4.2
CREATE TABLE categorie (
    id_categorie INT AUTO_INCREMENT PRIMARY KEY,
    libelle VARCHAR(40) NOT NULL UNIQUE
);

CREATE TABLE produit (
    id_produit INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(60) NOT NULL UNIQUE,
    prix DECIMAL(5,2) NOT NULL,
    stock INT NOT NULL DEFAULT 0,
    id_categorie INT NOT NULL,
    CONSTRAINT ck_produit_prix CHECK (prix > 0),
    CONSTRAINT ck_produit_stock CHECK (stock >= 0),
    CONSTRAINT fk_produit_categorie FOREIGN KEY (id_categorie) REFERENCES
categorie(id_categorie)
);

CREATE TABLE client (
    id_client INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(50) NOT NULL,
    prenom VARCHAR(50) NOT NULL,
```

```
    email VARCHAR(100) NOT NULL UNIQUE,
    telephone VARCHAR(15) NULL,
    ville VARCHAR(50) NOT NULL,
    date_inscription DATE NOT NULL
);

CREATE TABLE commande (
    id_commande INT AUTO_INCREMENT PRIMARY KEY,
    date_commande DATETIME NOT NULL,
    date_retrait DATE NOT NULL,
    statut ENUM('en_cours','prete','retiree','annulee') NOT NULL DEFAULT 'en_cours',
    montant_total DECIMAL(7,2) NOT NULL DEFAULT 0,
    id_client INT NOT NULL,
    CONSTRAINT ck_commande_retrait CHECK (date_retrait >= DATE(date_commande)),
    CONSTRAINT fk_commande_client FOREIGN KEY (id_client) REFERENCES client(id_client)
);

CREATE TABLE ligne_commande (
    id_commande INT NOT NULL,
    id_produit INT NOT NULL,
    quantite INT NOT NULL,
    prix_unitaire DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (id_commande, id_produit),
    CONSTRAINT ck_ligne_quantite CHECK (quantite > 0),
    CONSTRAINT fk_ligne_commande FOREIGN KEY (id_commande) REFERENCES commande(id_commande) ON
DELETE CASCADE,
    CONSTRAINT fk_ligne_produit FOREIGN KEY (id_produit) REFERENCES produit(id_produit)
);

-- Q1.4.3
SHOW TABLES;
DESCRIBE produit;
SHOW CREATE TABLE commande;
-- 5 tables. Dans produit, stock a bien 0 par défaut et id_categorie est NOT NULL.

-- Q1.4
-- ligne_commande a une clé étrangère vers commande. Si on supprime commande d'abord,
-- le SGBD refuse (erreur 1451) car ligne_commande pointe encore dessus.
-- On supprime d'abord les tables qui dépendent des autres.
```

Partie 2 : insérer, tester les contraintes, charger

```
USE fournil;

-- Q2.1
INSERT INTO categorie (libelle) VALUES ('Pains'), ('Viennoiseries'), ('Pâtisseries'),
('Snacking');
SELECT * FROM categorie ORDER BY id_categorie;
--le SGBD a mis 1, 2, 3, 4.

-- Q2.2
INSERT INTO produit (nom, prix, stock, id_categorie) VALUES
('Baguette tradition', 1.30, 40, 1),
('Pain de campagne', 3.20, 12, 1),
('Pain aux céréales', 3.60, 8, 1),
('Croissant', 1.20, 30, 2),
('Pain au chocolat', 1.30, 25, 2),
('Pain aux raisins', 1.50, 10, 2),
('Chausson aux pommes', 1.80, 3, 2),
('Éclair au chocolat', 2.90, 12, 3),
```

```
('Tarte au citron', 3.50, 0, 3),
('Flan pâtissier', 2.60, 6, 3),
('Sandwich jambon-beurre', 4.50, 15, 4),
('Quiche lorraine', 4.20, 5, 4);
SELECT COUNT(*) FROM produit;
-- Résultat : 12

-- Q2.3
INSERT INTO client (nom, prenom, email, telephone, ville, date_inscription) VALUES
('Martin', 'Léa', 'lea.martin@mail.fr', '0601020304', 'Lyon', '2026-09-02'),
('Dubois', 'Chloé', 'chloe.dubois@mail.fr', NULL, 'Lyon', '2026-09-10');
SELECT * FROM client;

-- Q2.4 (prévision /résultat)

-- a. prévision : accepté
INSERT INTO produit (nom, prix, id_categorie) VALUES ('Brioche', -2, 2);
-- Résultat : refusé, erreur 4025 CONSTRAINT ck_produit_prix failed -> RG2

-- b. prévision : refusé
INSERT INTO client (nom, prenom, email, ville, date_inscription)
VALUES ('Martin', 'Paul', 'lea.martin@mail.fr', 'Lyon', '2026-10-16');
-- Résultat : refusé, erreur 1062 Duplicate entry pour la clé email -> RG3

-- c. prévision : refusé
INSERT INTO produit (nom, prix, id_categorie) VALUES ('Macaron', 1.90, 9);
-- Résultat : refusé, erreur 1452 Cannot add or update a child row -> RG2

-- d. prévision : refusé
INSERT INTO commande (date_commande, date_retrait, id_client) VALUES ('2026-10-16 07:00:00',
'2026-10-15', 1);
-- Résultat : refusé, erreur 4025 CONSTRAINT ck_commande_retrait failed -> RG4

-- e. prévision : accepté
INSERT INTO commande (date_commande, date_retrait, id_client) VALUES ('2026-10-16 07:00:00',
'2026-10-16', 1);
SELECT * FROM commande;
-- Résultat : acceptée. Le SGBD a mis statut = en_cours et montant_total = 0.00

-- f. prévision : accepté
INSERT INTO ligne_commande (id_commande, id_produit, quantite, prix_unitaire)
VALUES ((SELECT MAX(id_commande) FROM commande), 1, 0, 1.30);
-- Résultat : refusé, erreur 4025 CONSTRAINT ck_ligne_quantite failed -> RG5

-- g. prévision : refusé
DELETE FROM categorie WHERE libelle = 'Pains';
-- Résultat : refusé, erreur 1451 Cannot delete or update a parent row -> RG2

-- h. prévision : accepté
INSERT INTO ligne_commande (id_commande, id_produit, quantite, prix_unitaire)
VALUES ((SELECT MAX(id_commande) FROM commande), 1, 2, 1.30);
-- Résultat : acceptée

-- Q2.4
-- a, d et f viennent d'une contrainte CHECK.
-- c et g viennent d'une clé étrangère, c vise une catégorie qui n'existe pas,
-- g veut supprimer une catégorie encore utilisée.

-- Q2.5.1
ALTER TABLE produit ADD allergenes VARCHAR(100) NULL;
UPDATE produit SET allergenes = 'gluten, lait, beurre' WHERE nom = 'Croissant';
```

```sql
-- Q2.5.2
UPDATE client SET telephone = '0699887766' WHERE email = 'lea.martin@mail.fr';

-- Q2.5.3
SELECT COUNT(*) FROM ligne_commande;
DELETE FROM commande WHERE id_commande = (SELECT id FROM (SELECT MAX(id_commande) AS id FROM
commande) AS t);
SELECT (SELECT COUNT(*) FROM commande) AS commandes, (SELECT COUNT(*) FROM ligne_commande) AS
lignes;
-- avant, 1 ligne de commande. Après, 0 commande et 0 ligne.
-- La ligne des 2 baguettes a été supprimée avec la commande : ON DELETE CASCADE (RG5).

-- Q2.5
-- sans WHERE, l'UPDATE change le téléphone de tous les clients.
-- Tout le monde aurait le numéro de Léa, et il n'y a pas d'annulation possible.

-- Q2.6
-- (exécution de fournil_donnees.sql)
SELECT (SELECT COUNT(*) FROM categorie) AS categories, (SELECT COUNT(*) FROM produit) AS
produits,
       (SELECT COUNT(*) FROM client) AS clients, (SELECT COUNT(*) FROM commande) AS commandes,
       (SELECT COUNT(*) FROM ligne_commande) AS lignes;
```

Partie 3 : interroger la base

```sql
USE fournil;

-- ---------- 3.1 Une seule table ----------

-- Q3.1
SELECT nom, prix
FROM produit
ORDER BY prix DESC, nom ASC;
-- c'est bon

-- Q3.2
SELECT nom, prenom, ville
FROM client
WHERE telephone IS NULL
ORDER BY nom;
-- c'est bon

-- Q3.3
SELECT id_commande, date_commande, statut
FROM commande
WHERE date_commande >= '2026-10-14' AND date_commande < '2026-10-15';
-- c'est bon

-- Q3.4
SELECT statut, COUNT(*) AS nb_commandes
FROM commande
GROUP BY statut;
-- c'est bon

-- ---------- 3.2 Jointures ----------

-- Q3.5
SELECT p.nom, p.prix
FROM produit p
INNER JOIN categorie c ON c.id_categorie = p.id_categorie
```

```sql
WHERE c.libelle = 'Viennoiseries' AND p.prix < 1.50;
-- c'est bon

-- Q3.6
SELECT p.nom, l.quantite, l.prix_unitaire, l.quantite * l.prix_unitaire AS sous_total
FROM ligne_commande l
INNER JOIN produit p ON p.id_produit = l.id_produit
WHERE l.id_commande = 9;
-- c'est bon

-- Q3.7
SELECT co.id_commande,co.statut, cl.nom , cl.prenom, cl.telephone
FROM commande co
INNER JOIN client cl ON cl.id_client = co.id_client
WHERE co.date_retrait = '2026-10-16'
--c'est bon

-- ---------- 3.3 Regroupements ----------

-- Q3.8
SELECT date_retrait, SUM(montant_total) AS chiffre_affaires
FROM commande
WHERE statut = 'retiree'
GROUP BY date_retrait;
--c'est bon

-- Q3.9
SELECT p.nom, SUM(l.quantite) AS quantite_totale
FROM ligne_commande l
INNER JOIN produit p   ON p.id_produit = l.id_produit
INNER JOIN commande co ON co.id_commande = l.id_commande
WHERE co.statut<> 'annulee'
GROUP BY p.nom
ORDER BY quantite_totale DESC
LIMIT 3;
--c'est bon

-- Q3.10
SELECT cl.nom,  cl.prenom ,COUNT(co.id_commande) AS nb_commande
FROM client cl
INNER JOIN commande co ON cl.id_client = co.id_client
GROUP BY cl.id_client
HAVING nb_commande >= 3;
--c'est bon

-- Q3.11
SELECT cl.ville, COUNT(DISTINCT cl.id_client) AS nb_clients, SUM(co.montant_total) AS
chiffre_affaires
FROM client cl
INNER JOIN commande co ON co.id_client = cl.id_client
WHERE co.statut = 'retiree'
GROUP BY cl.ville
ORDER BY chiffre_affaires DESC;
--c'est bon

-- ---------- 3.4 Jointures externes et sous-requêtes ----------

-- Q3.12
SELECT cl.nom, cl.prenom
FROM client cl
LEFT JOIN commande co ON co.id_client = cl.id_client
WHERE co.id_commande IS NULL;
```

```sql
--c'est bon

-- Q3.13
SELECT p.nom
FROM produit p
WHERE NOT EXISTS (
    SELECT 1 FROM ligne_commande l WHERE l.id_produit = p.id_produit
);
--c'est bon

-- Q3.14
SELECT ROUND(AVG(montant_total), 2) AS panier_moyen
FROM commande
WHERE statut = 'retiree';
--c'est bon

-- Q3.15
SELECT id_commande, montant_total
FROM commande
WHERE statut = 'retiree'
  AND montant_total > (SELECT AVG(montant_total) FROM commande WHERE statut = 'retiree')
ORDER BY montant_total DESC;
--pas trouver ( réponse de claude)

-- Q3.16
SELECT l.id_commande, p.nom, l.prix_unitaire AS prix_paye, p.prix AS prix_actuel
FROM ligne_commande l
INNER JOIN produit p ON p.id_produit = l.id_produit
WHERE l.prix_unitaire <> p.prix;
--pas trouver ( réponse de claude)

-- Question 3.16
-- Réponse : le croissant est passé de 1,10 € à 1,20 € le 15 octobre. Les commandes d'avant
gardent le prix payé ce jour-là.
-- Avec produit.prix, on recalculerait les anciennes commandes au nouveau prix :
-- le chiffre d'affaires des jours passés augmenterait alors qu'on n'a pas encaissé plus.

-- Q3.17 (attendu : aucune ligne)
SELECT c.id_commande, c.montant_total, SUM(l.quantite*l.prix_unitaire) AS total_ligne
FROM commande c
JOIN ligne_commande l ON l.id_commande = c.id_commande
GROUP BY c.id_commande
HAVING montant_total <> total_ligne;

-- ---------- 3.5 Une vue ----------

-- Q3.18 (attendu : 5 lignes dans la vue, 3 en cours)

-- Question 3.18
CREATE OR REPLACE VIEW v_a_preparer AS
SELECT co.id_commande, co.statut, co.montant_total, c.nom, c.prenom, c.telephone
FROM commande co
INNER JOIN client c ON c.id_client = co.id_client
WHERE co.statut IN ('en_cours', 'prete')
  AND co.date_retrait = '2026-10-16';

SELECT * FROM v_a_preparer WHERE statut = 'en_cours';
-- Résultat : la vue contient 5 lignes, dont 3 en cours (16, 17, 20)
-- Réponse :
```